



C프로그래밍 (CSE2003)

1. Structure of a C Program

표현식(Expression)

- 프로그램 언어에서 피연산자(operand)들과 연산자(operator)들로 구성된 식.

- Operator(연산자) : $=$, $+$, $-$, $*$, $/$ 등.

- Operand (피연산자): operator를 가지고 계산되는 상수 또는 변수.

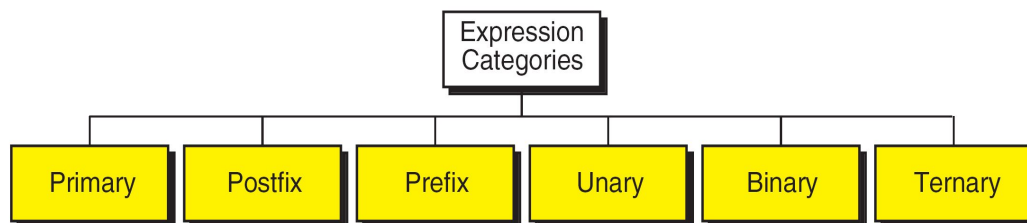
- 표현식(expression)은 항상 결과(반환) 값을 갖는다.

- 표현식의 예: $2 + 5$ 결과값: 7

- a 결과값:-a

- $2 + 5 * 7$ 결과값:37

- 표현식의 구분



연산자	연산의 예	의미
=	a=20	대입
+	4+3	덧셈
-	4-3	뺄셈
*	4*3	곱셈
/	4/3	나눗셈
%	4%3	나머지

표현식(Expression)

■ Primary expression

: operator가 없이 오직 하나의 operand로 이루어진 가장 기본적인 형태

□ 식별자(Identifier)

□ 식별자 예 : a, b12, price, calc, INT_MAX 등

□ 상수 (Constants)

□ Constants 예 : 5, 123.45, 'A', "Welcome" 등

□ 괄호식 (Parenthetical Expression)

□ 괄호식 예 : $(2*3)+4$, $(a=23+b*6)$ 등

a

Identifier

7

Constant

$(2 + a - 3)$

Expression

표현식(Expression)

<원의 넓이와 원주를 구하는 프로그램>

```
#include <stdio.h>

int main (void)
{
    float circ;
    float area; ➡ 식별자
    float radius;

    printf("\nPlease enter the value of the radius:");
    scanf("%f", &radius);

    circ = 2 * 3.14 * radius; ➡ 상수
    area = 3.14 * (radius * radius); ➡ 괄호식

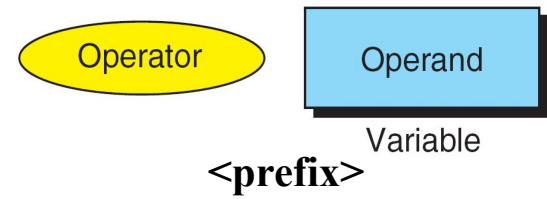
    printf("\nRadius is :      %10.2f", radius);
    printf("\nCircumference is :%10.2f", circ);
    printf("\nArea is :          %10.2f", area);

    return 0;
}
```

표현식(Expression)

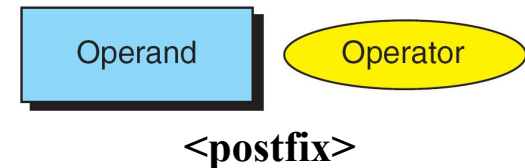
▪ Prefix expression

- 형식 : operator가 operand 앞에 나옴
- 예 : ++a, --a



▪ Postfix expression

- 형식 : operator가 operand 뒤에 나옴
- 예 : a++, a--



▪ ++a와 a++의 공통점

- $a = a + 1$ 의 결과와 같은 효과를 낸다.

표현식(Expression)

```
#include <stdio.h>

int main (void)
{
    int a;

    a = 4;
    ++a;
    printf("Value of ++a : %d\n", a);

    a = 4;
    a++;
    printf("Value of a++ : %d\n", a);

    a = 4;
    a = a + 1;
    printf("Value of a+1 : %d\n", a);

    return 0;
}
```

프로그램의 실행 결과는?

Value of ++a : 5

Value of a++ : 5

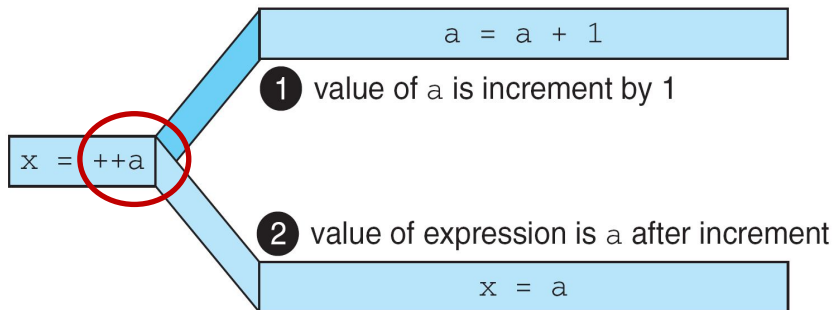
Value of a+1 : 5

표현식(Expression)

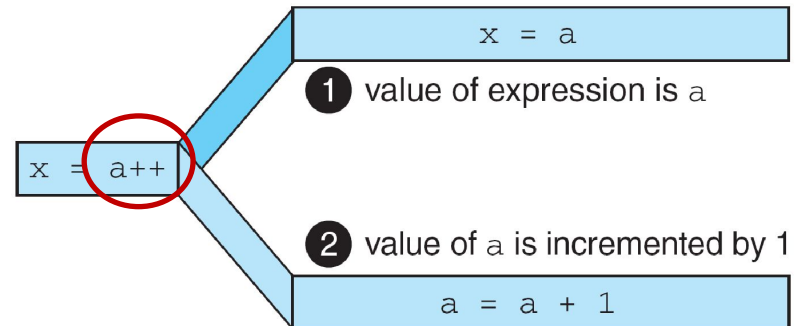
■ ++a 과 a++의 차이

- prefix와 postfix는 증가/감소 연산을 수행하는 시점이 다름
 - prefix의 경우, 변수의 참조보다 먼저 연산을 수행
 - postfix의 경우, 변수의 참조 후에 연산을 수행

<prefix>



<postfix>



표현식(Expression)

<prefix>

```
#include <stdio.h>
```

```
int main (void)
```

```
{
```

```
    int a, x;
```

```
    a = 4;
```

```
    printf("Value of a      :%d\n", a);
```

```
    printf("Value of ++a    :%d\n", ++a);
```

```
    printf("New value of a:%d\n", a);
```

```
    a = 4;
```

```
    x = ++a;
```

```
    printf("Value of x      :%d\n", x);
```

```
    return 0;
```

```
}
```

프로그램의 실행 결과는?

```
Value of a      :4
Value of ++a    :5
New value of a :5
Value of x      :5
```

<postfix>

```
#include <stdio.h>
```

```
int main (void)
```

```
{
```

```
    int a, x;
```

```
    a = 4;
```

```
    printf("Value of a      :%d\n", a);
```

```
    printf("Value of a++    :%d\n", a++);
```

```
    printf("New value of a:%d\n", a);
```

```
    a = 4;
```

```
    x = a++;
```

```
    printf("Value of x      :%d\n", x);
```

```
    return 0;
```

```
}
```

프로그램의 실행 결과는?

```
Value of a      :4
Value of a++    :4
New value of a :5
Value of x      :4
```


표현식(Expression)

■ 증가연산(prefix, postfix)과 일반산술연산(infix)의 비교

```
#include <stdio.h>

int main (void)
{
    int x = 10;

    printf("%d\n", x++);
    printf("%d\n", x);
    printf("%d\n", --x);
    printf("%d\n", x);

    return 0;
}
```



printf	X
	10
10	11
11	11
10	10
10	10

```
#include <stdio.h>

int main (void)
{
    int x = 10;

    printf("%d\n", x+1);
    printf("%d\n", x);
    printf("%d\n", x-1);
    printf("%d\n", x);

    return 0;
}
```



printf	X
	10
11	10
10	10
9	10
10	10

표현식(Expression)

■ 증감연산 사용 이유

- 증감연산자를 이용하면 프로그램 형태가 간결
- 기계어 코드와 일대일 대응되므로 실행속도가 개선

■ 주의점

- 연산자의 위치에 따라 evaluation value가 다르므로 주의 요구
- 증감연산자는 ++, -- 자체가 연산자 기호 □ 중간에 공백이 들어가면 안됨
- 증감연산자는 피연산자로 변수를 사용할 수 있지만, 상수나 일반 수식을 피연산자로 사용 불가능

표현식(Expression)

문제) 잘못된 표현식은?

```
#include <stdio.h>

int main (void)
{
    int a, b, c;

    a = 7;
    b = 77++;          /* 상수에는 증가 연산자를 사용할 수 없다 */
    c = ++a;
    c--;
    --(a * c - 1);      /* 일반 수식에는 증가 연산자를 사용할 수 없다 */
    (b + a)++;         /* 일반 수식에는 증가 연산자를 사용할 수 없다 */

    return 0;
}
```

표현식(Expression)

■ Unary expression

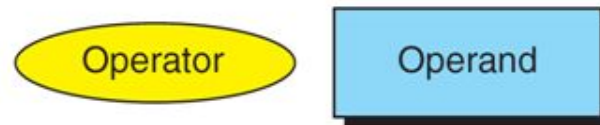
- 형식 : 연산자 – 피연산자 (operator - operand)

- 예

- +a, -a

- sizeof(int)

- (float)x 등.



<Unary expression>



<Binary expression>

■ Binary expression

- 형식 : 피연산자 – 연산자 – 피연산자 (operand – operator - operand)

- Operator가 operand사이에 있음

- 예

- a+7, 3+4, b-11 등 □ 덧셈 연산자(+, -)

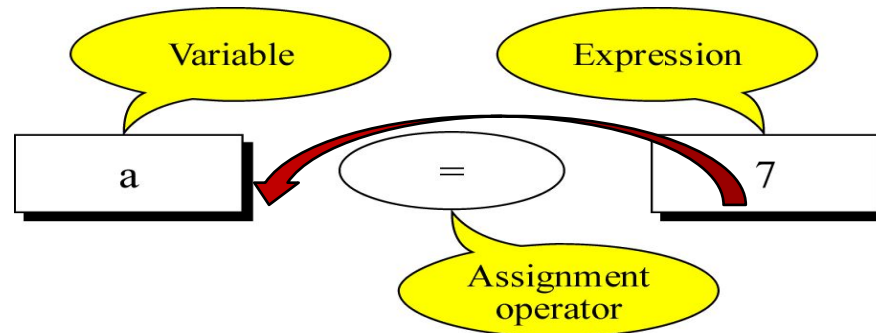
- 10*12, a /4, 5%2 등 □ 곱셈 연산자(*, /, %)

- %(modulo)는 나머지를 구하는 연산자

실습2

Assignment expressions

- 수학에서는 “우측의 값과 좌측의 값이 같다”라는 의미이나,
C 언어에서는 “우측의 값을 좌측의 저장 장소에 저장하라”라는 의미.
- assignment operator ‘=’의 좌측은 반드시 **single variable**이어야 한다.



□ 분류

□ simple assignment

□ 예 : $a=5$, $b=x+1$, $i=i+1$ 등

□ Compound assignment

□ 예) $x+=y$, $x*=y$, $x/=y$ 등

Contents of Variable x	Contents of Variable y	Expression	Value of Expression	Result of Expression	Compound Expression	Equivalent Simple Expression
10	5	$x = y + 2$	7	$x = 7$	$x += y$	$x = x + y$
10	5	$x = x / y$	2	$x = 2$	$x /= y$	$x = x / y$
10	5	$x = y \% 4$	1	$x = 1$	$x \&= y$	$x = x \& y$
					$x -= y$	$x = x - y$

Assignment expressions

▪ Destination of an assignment

- 대입 명령은 항상 대입 연산자(assignment operator)의 오른쪽에서 왼쪽으로.
`#include <stdio.h>`

<원의 넓이와 원주를 구하는 프로그램>

```
int main (void)
{
    float circ;
    float area;
    float radius;
    float temp1, temp2;

    radius = 9;
    9 = radius;

    circ = 2 * 3.14 * radius;
    circ / 2 = 3.14 * radius;

    temp1 = radius * radius;
    temp2 *= 3.14;

    area = temp2;
    temp2 = area;

    temp1 = temp2 = 0;
    temp1 = (temp2 = 0);

    return 0;
}
```

```
counter = 3;
counter = counter + 1;
counter = 5 * counter;
```



/*변수 원주를 구한 결과값을 변수 circ에 대입*/
/***circ/2는 변수가아닌 표현식이므로 잘못된 표현!***/

따라서 아래의 두 대입 명령문은
다른 의미를 갖는다. 같다는 표현을
하기 위해서는 “==“ 연산자 이용!

/*변수 temp2가 갖는 값을 변수 area에 대입*/
/*변수 area가 갖는 값을 변수 temp2에 대입*/

대입 명령은 항상 연산자의 오른쪽에서 왼쪽으로
대입되므로 옆의 두 명령은 같은 의미를 갖는다.
예 : `A = B = C = 0;` or `A = ( B = ( C = 0 ) );`

실습3

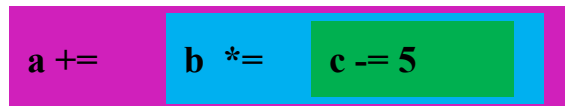
Precedence & Associativity

- **Precedence** : 다른 연산자들 사이의 우선 순위
 - 수학에서도 곱하기나 나누기 연산자가 더하기나 빼기 연산자보다도 우선 순위가 더 높은 것처럼 C언어에서도 연산자 사이에 우선순위가 있다.
 - Precedence의 예 :
 - $2 + 3 * 4$ □ $(2 + (3 * 4))$
 - $-b++$ □ $-(b++)$

- **Associativity** : 같은 순위의 연산자들 사이의 우선 순위
 - Associativity의 예 :
 - **Left-to-Right (Simple Assignment)**
 - $3 * 8 / 4 \% 4 * 5$ □ $((((3 * 8) / 4) \% 4) * 5)$



- **Right-to-Left (Compound Assignment)**
 - $a += b *= c -= 5$ □ $(a += (b *= (c -= 5)))$
 - $(a = a + (b = b * (c = c - 5)))$



Precedence & Associativity

Precedence and Associativity of Operators in C

Precedence	Associativity	Operators			
1	$L \rightarrow R$	()	[]	->	.
2	$R \rightarrow L$	!	~	++	--
		(type)	- (unary)	+ (unary)	sizeof
		* (dereference)	& (address of)		
3	$L \rightarrow R$	* (multiply)	/	%	
4	$L \rightarrow R$	+	-		
5	$L \rightarrow R$	<<	>>		
6	$L \rightarrow R$	<	<=	>	>=
7	$L \rightarrow R$	==	!=		
8	$L \rightarrow R$	& (bitwise and)			
9	$L \rightarrow R$	^			
10	$L \rightarrow R$				
11	$L \rightarrow R$	&&			
12	$L \rightarrow R$				
13	$R \rightarrow L$	? :			
14	$R \rightarrow L$	=	*=	/=	%=
		+=	--=	<<=	>>=
		&=	=	^=	
15	$L \rightarrow R$	,			

Precedence & Associativity

```
#include <stdio.h>

int main (void)
{
    int a = 10;
    int b = 20;
    int c = 30;

    printf("a * b + c is      :%d\n", a * b + c);
    printf("a * (b + c) is      :%d\n", a * (b + c));
    printf("3 * -8 + 34 % 5 is :%d\n", 3 * -8 + 34 % 5);

    return 0;
}
```

출력되는 결과는?

```
a * b + c is      :230
a * (b + c) is      :500
3 * -8 + 34 % 5 is : -20
```

형 변환(Type Conversion)

▪ Type Conversion

- 일반적으로 C 언어의 연산식에서 여러 피연산자의 자료형이 서로 다른 경우, 하나의 통일된 자료형으로 자동 변환하여 연산을 수행
- Implicit type conversion (coercion) :
 - C 컴파일러가 판단하여 자동으로 데이터형을 변형하는 경우.
 - 다음과 같은 경우 C 컴파일러가 판단하여 자동 형 변환을 수행한다.
 - 수식에서 데이터형이 혼합되어 사용되었을 때 값을 변환.
 - 특정한 데이터형의 변수에 다른 데이터형의 값을 대입할 때, 값을 변환.

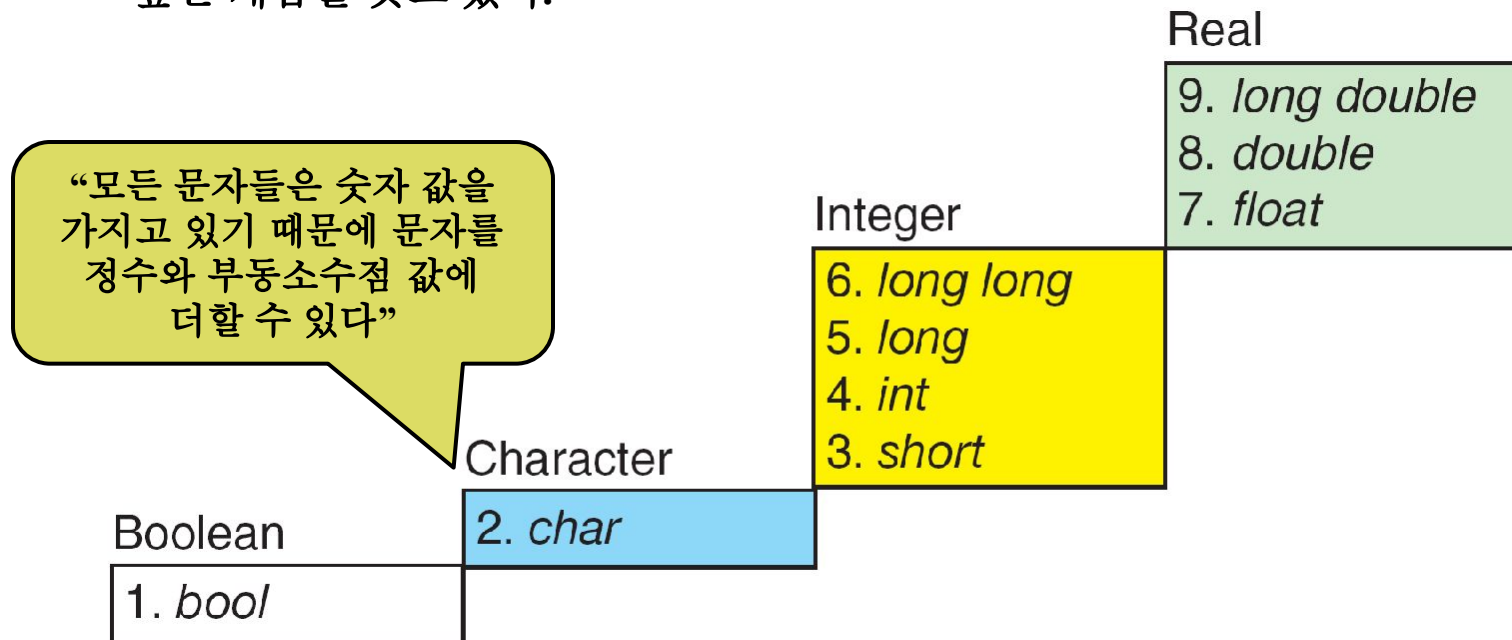
```
int x = 1;  
char y = 'a';  
x = y; //x = 97 (character c의 값 'a'의 ASCII code 값)
```

- Explicit type conversion (cast) : 프로그래머가 형 변환자(cast)를 사용하여 강제적으로 변형하는 경우.

```
float x = 1123.345;  
int y;  
y = (int)x; // y = 1123 (int형으로 변환되어 소수점 아래를 버린다.)
```

Implicit type conversion

- 서로 다른 데이터 형이 혼합하여 사용될 때
 - 수식에서 형 변환은 보통 범위가 큰 데이터형으로 변환
 - **Conversion Rank**
 - 각각의 타입들에 대해 계급을 할당
 - Ex) *long double* 타입의 실수는 *long* 타입의 정수보다 높은 계급을 갖고 있다.



Implicit type conversion

■ 형 확장(Promotion)

- 하나 이상의 데이터 형이 혼합식을 구성할 때
- 가장 큰 데이터 형에 맞춰 자동으로 형 변환이 이루어지는 규칙을 적용

```
#include <stdio.h>
```

```
int main (void)
{
```

```
    char ch;
    int i;
    float f;
    double d;
```

```
    ch = 'A';
    i = 10;
    f = 1.5;
    d = ch * i * f + 300;
```

```
    printf("result = %lf\n", d);
```

```
    return 0;
```

```
}
```

<실행결과>

```
kimjihwan@cspro:~/lec2_2$ vi implicit_type_conversion.c
```

```
kimjihwan@cspro:~/lec2_2$ gcc -o implicit_type_conversion implicit_type_conversion.c
```

```
kimjihwan@cspro:~/lec2_2$ ./implicit_type_conversion
```

```
result = 1275.000000
```

```
kimjihwan@cspro:~/lec2_2$
```

1) $ch * i$ □ ch 는 int형으로 확장되어 i 와 계산 수행된다.
여기서 ch 는 대문자 A 인데 이는 ASCII 코드값 65 로
계산된다.

2) $* f$ □ 1)에서 계산된 결과는 float형으로 확장되어
 f 와 계산 수행된다.

3) $+ 300$ □ 300도 float형으로 확장되어 2)의 결과에
더해진다.

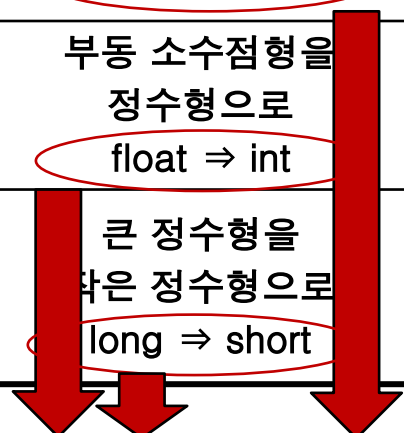
4) 계산의 결과는 double형으로 확장되어 d 에 대입된다.



Implicit type conversion

- 범위가 큰 데이터 형에서 작은 데이터 형으로의 변환은 문제 발생

형 변환	문 제 점
큰 부동 소수점형을 작은 부동 소수점형으로 <u>double $\Rightarrow$ float</u>	정밀도(유효 숫자)에 손실이 있다. 원래 값이 변환 데이터형의 범위를 벗어나는 경우의 결과는 예측 불허이다.
부동 소수점형을 정수형으로 <u>float $\Rightarrow$ int</u>	소수부를 잃어버리게 된다. 원래 값이 변환 데이터형의 범위를 벗어나는 경우는 결과는 예측 불허이다.
큰 정수형을 작은 정수형으로 <u>long $\Rightarrow$ short</u>	원래 값이 변환 데이터형의 범위를 벗어나는 경우에 일반적으로 하위 바이트들만이 복사된다.



```
long x = 3458234;  
short y;  
y = x; // y = -13108 (?)
```

Explicit type conversion(Cast)

▪ Explicit conversion (casting, cast operation)

- 프로그래머가 강제로 변수의 데이터형 변환 가능
- 피연산자의 값을 앞 위치의 괄호 안에서 지정한 자료형으로 변환하는 연산

```
#include <stdio.h>
```

```
int main (void)  
{
```

```
    double r1, r2;  
    int result1, result2;
```

```
    r1 = 3.4;  
    r2 = 2.1;
```

```
    result1 = r1 * r2;
```

```
    result2 = (int)r1 * (int)r2;
```

```
    printf("result1 = %d\n", result1);
```

```
    printf("result2 = %d\n", result2);
```

```
    return 0;
```

```
}
```

<실행결과>

```
kimjihwan@cspro:~/lec2_2$ vi explicit_type_conversion.c  
kimjihwan@cspro:~/lec2_2$ gcc -o explicit_type_conversion explicit_type_conversion.c  
kimjihwan@cspro:~/lec2_2$ ./explicit_type_conversion  
result1 = 7  
result2 = 6  
kimjihwan@cspro:~/lec2_2$
```

xpression

의 값을 type의
강제로 변환.

implicit type conversion

cast 연산자에 의한 explicit conversion



Explicit type conversion(Cast)

- 시스템, OS (운영체제), 컴파일러의 종류에 따라 각 데이터형의 메모리 저장 크기가 다를 수 있음
- 프로그램이 사용하는 데이터형의 크기를 알 수 있는 방법으로 sizeof 연산자를 사용할 수 있음 (unary expression)
 - sizeof (type)
 - 위의 연산자를 사용하면 해당 데이터형의 크기를 byte 단위로 알려줌

Ex) sizeof(int);
 sizeof(float);

cf) 시스템에 따라 데이터형의 크기가 다를 수 있다는 점은, 한 환경에서 다른 환경으로 프로그램을 가져갈 때 문제를 일으킬 수 있다.
 sizeof() 연산자는 데이터형의 크기가 중요시 되는 프로그램에서 유용하게 사용될 수 있다.

Statements

- 프로그램에 의해서 수행되는 하나의 동작(action)
- 직접적으로 하나 또는 그 이상의 실행 가능한 컴퓨터 명령어로 변환됨

▪ Statement의 종류

▣ Null statement

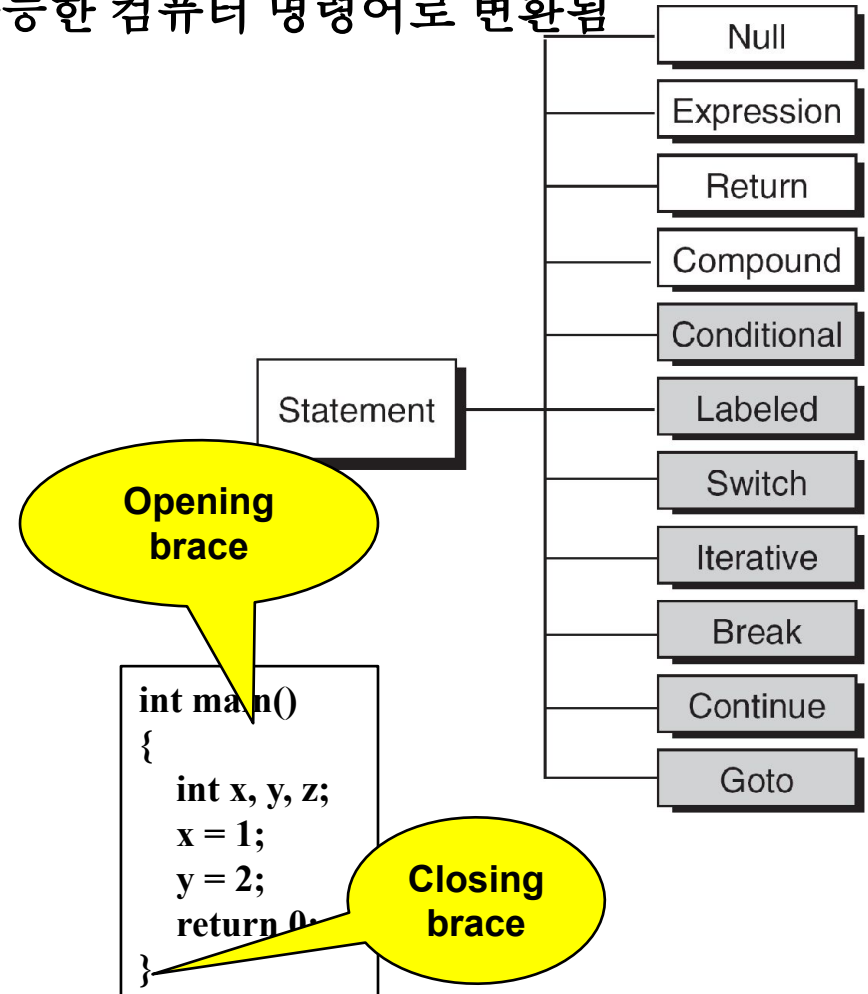
- ▣ ; □ statement가 있어야 하지만 아무런 동작도 필요하지 않을 경우 사용
- ▣ loop등에서 유용하게 사용
예) for(;;)

▣ Expression statements

- ▣ 표현식 뒤에 세미콜론(;)을 붙여 statement로 변환
- ▣ ‘;’는 컴퓨터에게 statement의 끝임을 나타냄

▣ Compound statements

- ▣ 0 (zero) 또는 그 이상의 문장으로 구성되는 코드 단위 : Block
- ▣ main 함수를 만들 때도 compound statements 사용
- ▣ 세미콜론(;)이 필요하지 않음



세 정수의 합, 평균, 분산을 구하는 프로그램

```
#include <stdio.h>
```

```
int main (void)
```

```
{
```

```
    int a, b, c;
```

```
    int sum;
```

```
    float mean;
```

```
    double variance;
```

```
    printf("Enter first integer:");
```

```
    scanf("%d", &a);
```

```
    printf("Enter first integer:");
```

```
    scanf("%d", &b);
```

```
    printf("Enter first integer:");
```

```
    scanf("%d", &c);
```

```
    sum = a + b + c;
```

```
    printf("Sum is : %d\n", sum);
```

```
    mean = (float)sum / 3;
```

```
    printf("Mean is : %f\n", mean);
```

```
    variance = ((a-mean)*(a-mean) + (b-mean)*(b-mean) + (c-mean)*(c-mean))/3.0;
```

```
    printf("Variance is : %lf\n", variance);
```

```
    return 0;
```

```
}
```

associativity, precedence, type conversion이
어떻게 적용되는지 살펴보자.

세 정수의 합, 평균, 분산을 구하는 프로그램

- 예) `int sum = 2;` 일 때,

- `mean = (float) sum / 3.0F;`

- cast 연산자에 의한 explicit conversion : `int -> float`
- 결과값 : `mean = 0.666666...`

: 수행 후, `sum`의 값은? 2

- `mean = sum / 3.0F;`

- implicit type conversion : `int -> float`
- 결과값 : `mean = 0.666666...`

: 수행 후, `sum`의 값은? 2

- `mean = (float) sum / 3;`

- cast 연산자에 의한 explicit conversion : `int -> float`
- implicit type conversion : `int -> float`
- 결과값 : `mean = 0.666666...`

: 수행 후, `sum`의 값은? 2

- `mean = sum / 3;`

- Implicit type conversion : `int -> float`
- 결과값 : `mean = 0.0`

: 수행 후, `sum`의 값은? 2

세 정수의 합, 평균, 분산을 구하는 프로그램

- 예) `int sum = 2;` 일 때, (계속)

- `mean = (float) (sum / 3);`

- cast 연산자에 의한 explicit conversion : `int -> float`
- 결과값 : `mean = 0.0`

: 수행 후, `sum`의 값은? 2

- `mean = sum / 3 * 100;`

- implicit type conversion : `int -> float`
- 결과값 : `mean = 0.0`

: 수행 후, `sum`의 값은? 2

- `mean = 100 * sum / 3;`

- implicit type conversion : `int -> float`
- 결과값 : `mean = 66.0`

: 수행 후, `sum`의 값은? 2

실습4

실습4-1